

Supplementary Material for *Declared Ambiguity: Fail-Closed Execution for Non-Idempotent Legacy APIs Without Idempotency Keys*

Anonymous Author(s)

1 PURPOSE AND STATUS

This document holds material referenced from the main paper but not required to follow its argument: the probe detail behind one negative result, the per-session tables of the uncontrolled hard-Redis-kill cell, the enumerated coverage gaps of the fault-injection matrix, and the measurement incidents found during collection. Sections appear in the order the main paper reaches them. It is submitted as supplemental material rather than as an appendix, for the reason recorded at the top of this file.

Every number here is generated by the same script that generates the paper's, from the same frozen results, and is checked by the same gate. Section names in the main paper are given in words rather than as cross-references, because the two documents are compiled separately.

2 FIGURES AND TABLES REFERENCED FROM THE MAIN PAPER

Three floats live here rather than in the manuscript, each with a pointer left in the section that used it. They are reproduced without change; only their location moved.

2.1 The trade, per system

Referenced from the main paper's RQ1 subsection, which states the result in prose and points here for the picture.

2.2 Declared ambiguity by crash point

Referenced from the main paper's RQ1 subsection, which keeps the two cells that carry the argument — `before_intent_write` at zero and `after_barrier_before_dispatch` at its maximum — and points here for the full decomposition across all six crash points and three capability classes.

2.3 Selected design assumptions, by the three questions

Referenced from the main paper's related-work section, whose three subsections are this table's rows in prose.

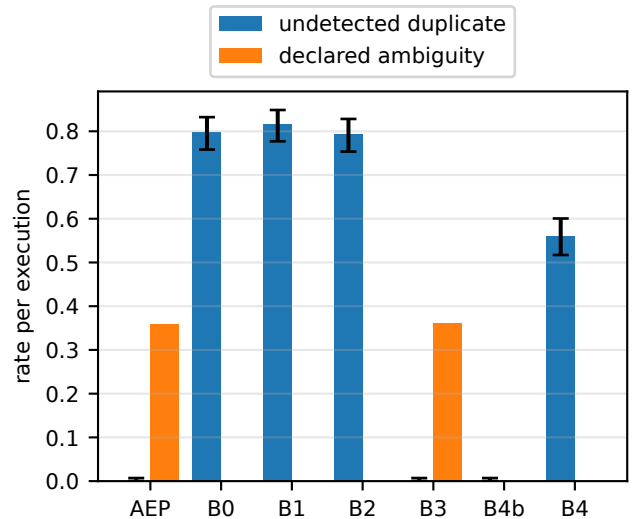


Fig. 1. The trade, per system. Crashed regime only, pooled over crash points and endpoint capabilities weighted by executions. The three systems on the right have an empty left bar; only AEP-full and its ablation B3 have a right bar at all. Interval is Wilson on the pooled counts — the cluster-aware bootstrap intervals, which are wider, are in `per-cell-metrics.csv` and are the ones to quote.

3 WHY THE PROVABLY-EMPTY CELL IS NOT RESOLVED, AND WHAT CLOSING IT WOULD COST

The paper's RQ1 subsection reports the negative result — the `after_barrier_before_dispatch` cell is provably empty in the world and not resolvable from the store, so recovery fails closed — and points here for the probe detail and the price of closing it.

3.0.0.1 Why the store cannot tell the two histories apart: The last durable write before the external call is the dispatch authorization. The runner then performs preflight, mints the capability and calls the connector *without touching Redis again*. A recovery process reading the store afterwards sees an authorized, unresolved intent — and that is exactly what it would see had the worker died *during* transmission,

TABLE 1

AEP-full’s declared-ambiguity rate, by crash point and endpoint capability. The rate is not a constant of the protocol: it is zero where the crash precedes the intent write (nothing was promised), zero throughout AUTH (absence is provable), and highest exactly where no effect can exist but the endpoint cannot say so. Undetected duplicates and lost effects are 0 in every cell of this table.

crash point	AUTH	POS-ONLY	NONE
before_intent_write	0/30	0/30	0/30
after_intent_			
before_barrier	0/30	30/30	30/30
after_barrier_			
before_dispatch	0/30	30/30	30/30
mid_dispatch	0/30	0/30	30/30
after_response_			
before_resolution	0/30	1/30	30/30
after_resolution_			
before_barrier	0/30	2/30	10/30

with the request already on the wire. The two histories are identical in everything durable, so no amount of reasoning over the store separates them, and failing closed is the correct behaviour given what recovery can observe rather than a conservatism that better engineering would remove.

3.0.0.2 What closing it would cost: A third durable barrier immediately before transmission would split the two cases and let recovery refute the earlier one. The measured price of a barrier on this configuration is 983.3ms, so that is a 24.6% increase in step latency to convert one crash point’s declared ambiguities into confirmed non-events. It also *narrows rather than closes* the window: a crash between that barrier and the socket write returns the same ambiguity, so the trade buys a smaller residual and not the absence of one. We record the trade rather than take it. Its value depends entirely on a deployment’s ratio of crash points, which our uniform matrix deliberately does not model, so we have no basis on which to recommend it either way.

4 THE UNCONTROLLED HARD-REDIS-KILL CELL, PER SESSION

The paper’s prevention subsection keeps the replication claim and its direction, and points here for the per-session tables. Nothing in this section is load bearing for the paper’s argument: the controlled fault carries the result, and this cell is retained as the replication that agrees in direction at a magnitude the uncontrolled injector sets.

4.0.0.1 The magnitude moves across sessions: The cell was re-collected 4 further times under an identical configuration — same cell identity hash, same per-run seed, same pinned Redis image — for 120 executions per arm. AEP-full’s unwanted-applied count ranged 4–20 out of 30; B3 recorded 28/30 in every one, a range of 0. Taking the session rather than the execution as the unit, the difference averaged 17.2 effects, 95% interval [6.1, 28.4], and the 18 of the table above falls inside it.

4.0.0.2 The variation has a candidate cause that this design cannot establish: Taking the session as the unit, the four sessions separate the runs that applied an effect from those that did not by +70, -4, +282, +74ms: mean +105ms, 95% interval [-91, +302]. **The half-width is 196ms, 1.86**

times the mean it brackets, and the interval contains zero. We give the four session differences individually because the summary is what fails here: the interval admits effects three times the mean and effects of the opposite sign, and this design does not distinguish them. We give no p , and we do not put a count of sessions on one side of zero in place of one. Over 4 sessions a two-sided sign test cannot return below 0.125, so no outcome this design admits could have reached significance.

4.0.0.3 B3 is not the negative control this paper previously reported: Pooled over the same 120 executions it read as a clean null, and pooling is what made it read that way. At the session level it is uninformative: -28, +27, -31, -152ms, mean -46ms, interval [-166, +74], a half-width of 120ms and 2.61 times its own mean. A comparison that wide could not have contradicted the mechanism whatever B3 had done, so its failure to contradict it was never evidence for it.

The original cell was collected on a different filesystem from the four replications, a difference no run configuration records, so we report it separately: it agrees in direction at 88ms over 30 runs ($p = 0.03$), with its own control at 26ms ($p = 0.53$, $n = 30$). **It is one session: it pools nothing, and it replicates nothing.**

5 COVERAGE OF THE FAULT-INJECTION MATRIX

The main paper’s external-validity subsection states the headline coverage and points here. The full plan is 1068 runs; we collected 432, comprising 3780 executions over 126 cells, chosen to answer the research questions rather than to fill the grid. The run count for every cell is in `experiments/results/matrix/MANIFEST.md` and in `analysis/coverage.json` in the artifact.

What that leaves uncovered, precisely — six named gaps, of which three have since been closed and are recorded here with what closing them changed:

- **B4’s and B4b’s AUTH cells were incomplete and are now collected — and completing them corrected the value.** In an earlier draft B4’s AUTH cell held twenty executions and B4b had none, so that cell printed as a dash. Both are now $n = 180$ and $n = 180$ over all six crash points, the same shape as every other cell in the paper’s outcomes table, and that table is free of dashes. We record what the completion changed, because it is a caution about partial cells generally and not only about this one: the earlier twenty executions came from a *single* crash point, so the figure that draft quoted was not a wide interval around the value the full cell reports — it was one crash point standing in for six, and the six differ enough that none of them represents the pooled rate. The full cell puts B4’s AUTH duplicate rate at 0.5278, in line with its other two classes and far from the figure the partial cell suggested. The direction of the argument is unchanged, which is why we could describe the gap as non-essential before collecting it, but the number we quoted for it was wrong, and a partial cell being unrepresentative rather than merely imprecise is the more general lesson. Every other cell in the outcomes table spans all the crash

TABLE 2

Selected design assumptions, organised by the three questions of the main paper’s related-work section. “Endpoint must cooperate” denotes idempotence, an idempotency key, transaction enrolment, or compensation. The rows are not ordered by guarantee strength; they expose different residuals when their stated assumptions fail.

	endpoint must cooperate	can report <i>unknown</i>	residual on failure
Lease / fenced CAS [1], [2]	n/a	no	duplicate effect
Two-phase commit [3], [4]	prepare and vote	no	blocked participant
Sagas [5], [6]	compensation	no	compensation gap
Transactional outbox [7]	idempotent consumer	no	duplicate delivery
Idempotency-Key header [8]	key support	no	duplicate effect
Locks with intent [9]	effect inside storage	no	—
Dataflow snapshots [10], [11]	transactional sink	no	duplicate external effect
Beldi [12]	yes	no	—
Durable Functions [13]	yes	no	duplicate or loss
Boki [14]	yes	no	—
ExoFlow [15]	yes	no	—
Temporal-style, ∞ retries [16]	yes	no	duplicate effect
Temporal-style, 1 attempt [16]	yes	no	lost effect
AEP	no	yes	declared ambiguity

TABLE 3

The *uncontrolled* hard-Redis-kill ablation — the original `docker kill` cell. Identical faults; the only difference is whether the system waits for the durability acknowledgement before dispatching. The rate is over executions, one per run. Source: `redis-kill-ablation.csv`, regime `redis-kill-preack`, `NO-READBACK`.

	runs	unwanted count	applied effect rate	canary survived
AEP-full	30	10	0.3333	30/30
B3 (no barrier)	30	28	0.9333	30/30

points that apply to its system: six for the systems that run `aep_core`’s workflow, and five for B0–B2, which have no window between writing a record and acknowledging it durable because they write no record.

- **The prevention result no longer rests on one cell; it still rests on one host.** The original hard-Redis-kill ablation was collected on `NO-READBACK` only, at one crash point, at 30 runs per system, and was the whole of the barrier’s measured case. The paper’s prevention subsection now leads with a controlled-fault collection over 3 sessions and all 3 capability classes, whose between-session spread is 1 count per class. What has not changed is the machine: every collection here comes from one host, so the effect size’s dependence on host timing is now measured *on* that host rather than established *across* hosts, and a second machine would still be sampling something we have not observed.

We predicted that the other two capability classes would move the *declared-ambiguity* column and not the *applied-effect* column, because whether an effect reached the provider is not something a read-back capability can change after the fact. We since collected those cells: 4 pre-registered sessions, 30 runs

per arm per session, run-level interleaved, with the analysis fixed in advance. **The applied-effect column moved, and it moved in both directions.** The `AUTHORITATIVE-READBACK` minus `NO-READBACK` applied-rate difference ran +0.0, -10.0, +23.3 and +36.7 percentage points across the 4 sessions — a spread from -10.0 to +36.7 points on a comparison we expected to sit at zero. That is our own prediction contradicted in our own data, and we report it rather than resolving it.

The session-clustered interval on that difference contains zero, at [-21.4, +46.4] percentage points. **It contains zero because the sessions disagree with one another, not because the effect is absent:** the half-width is 33.9 points, wider than the mean it brackets at +12.5 points, so this design could not have resolved the effect it observed. We registered a minimum detectable effect in advance and do not quote it here, because it was derived without a between-session variance component and so understates the precision this question needed; the comparison we make instead is the half-width against the observed mean. **What we report is therefore a failure to reject at a precision inadequate to the question — not evidence that capability class leaves the applied-effect column alone.** The argument above is no longer unsupported: it is contradicted in the data, and untested at adequate power.

- **The in-flight Redis-kill variant** (kill during transmission rather than before acknowledgement) **is now collected, and it ties.** Over 2 sessions the two arms differ by at most 1 count in any capability class, both at ceiling: the tie our analysis predicted, measured rather than inferred. Two qualifications travel with it. The second session is a *deterministic replay* of the first rather than an independent draw — the harness assigns seeds per cell and repetition, so where no

race remains the second collection reproduces the first exactly, which is itself the clearest evidence that none does. And B3's lowest cell is 27/30, exactly the floor below which we would have read the result as a mis-timed injector rather than as a tie; it is the boundary, not headroom.

- **The 30% crash-probability regime** is implemented and not collected, so every rate in the paper comes from either an all-crash or a no-crash regime and nothing in between.
- **The alternative read-back keying** is a sensitivity variant that has not been run; all results use one keying.
- **One run did not complete** and is listed as such in the manifest rather than dropped silently.

A rate we did not measure is absent from the paper's tables rather than interpolated, and the manifest shipped with the results states the run count for every cell so a reader can see the shape of the coverage without reconstructing it.

6 MEASUREMENT INCIDENTS FOUND DURING COLLECTION

The main paper's internal-validity subsection states each of these in one sentence and points here. All three are defects in the *measurement*, found by the fault-injection matrix rather than by the unit suite, and all three are fixed; they are recorded because the general point — that a fault-injection harness is a stronger verification tool than a unit suite — cuts both ways, and there may be defects it has not found.

6.1 A resume defect that would have inflated counts

Our matrix runner supports resumption, and a run whose previous attempt was interrupted left event shards on disk that a fresh attempt could merge into its own log, double-counting executions that were never part of the run being recorded. It surfaced as a configuration-digest mismatch — the digest check did what it exists to do — but between two attempts of the same harness version the merge would have been *silent*, and the error class is “a resumed run over-counts”, which for a paper measuring rates is as bad as errors get. The safeguard is now explicit: the runner discards stale event shards and any stale summary before a run writes anything, logs what it discarded, and leaves the run's *inputs* — the rendered provider config and the oracle ledger — untouched, with a test pinning that distinction. Every resumed run in the current results was collected after the fix; runs collected before it were re-analysed and the single affected run was re-collected.

6.2 Two further defects the matrix found that the unit suite had not

A re-executing supervisor abandoned the lease instead of waiting for it, and execution-state seeding was not idempotent. Both made a *baseline* look better than it is, so both were biasing against our own hypothesis, and both are fixed.

6.3 The test suite is destructive to a running matrix

While collecting the B4/B4b cells we ran the project's own integration suite against the same Redis. The suite is deliberately destructive: it deletes the `aep:*` namespace in the database the matrix is using, and one of its durability tests `docker restarts` the matrix's Redis container by name to prove the AOF survives. Several runs died with connection errors, and the server the batch was collecting against was replaced underneath it.

Runs that *fail* are harmless — they write no summary, and the resumption logic re-runs them. The runs that *completed* during that window are the hazard, because a Redis blip makes a re-executing baseline retry, a baseline that retries more produces more duplicates, and that bias points toward our own hypothesis. They are also indistinguishable from clean runs in every artifact the analysis reads. Every B4/B4b run collected in the disturbed window was therefore discarded and re-collected on an otherwise idle host, and the server's `run_id` was recorded before and after the replacement batch to confirm it was not restarted again; none of the discarded runs contributes a number to the paper.

Two rules follow, and the artifact enforces both rather than documenting them: the runner refuses to start beside a host-level fault injector, and it now records the server's `run_id` with every run so that a restart during collection is visible in the results rather than inferred from a connection error that happened to be logged. We report the episode because the generalisation is not “be careful”: it is that a fault-injection harness and a fault-injection test suite make the same assumptions about owning the system, and running both is a measurement error that produces ordinary-looking data.

7 THE BARRIER IS A DEPLOYMENT CHOICE, WITH THREE MEASURED POINTS

The $\approx$ 2-second figure above is the one a reader would carry away as “AEP's overhead”. It is not: it is the price of one of the protocol's two guarantees, under one of the durability settings that guarantee can be bought at. Table 4 is the decision as a deployment actually faces it.

Three points, and the third is the one the earlier framing of this paper concealed.

7.0.0.1 *everysec* with the barrier: The default, and the expensive one. An acknowledgement must wait for the next scheduled `fsync`, up to a second away, and the protocol waits twice.

7.0.0.2 *always* with the barrier: The identical crash-free cells — same systems, endpoint, regime, seeds, host and provider — against a Redis whose only changed line is `appendfsync always`. The barrier's cost is -9.2ms [-27.2, 44.6]ms over fifteen runs per arm: negative, and not distinguishable from zero. Under *always* the `fsync` has already happened by the time the write returns, so `WAITAOF` has nothing left to wait for.

That is the measurement this section turns on. The barrier does not become cheap under *always*; it costs nothing, and the 1939.7ms it costs under *everysec* is therefore the price of the `fsync` boundary rather than of the barrier. A protocol step under *everysec* waits for the next scheduled

TABLE 4

The barrier is a deployment choice, not a fixed cost. All three rows run the same pre-dispatch intent ledger and therefore make the same detection claim; they differ in what they pay to also withhold dispatch when durability cannot be confirmed. The barrier column is each row’s own median minus a B3 median collected under the same `appendfsync` policy. Crash-free runs only, E5-gated, 2 000 ms provider floor.

configuration	median (ms)	over floor	barrier (ms)	prevents	claim
AEP-full, <code>everysec</code>	4 004.9	2 004.9	1 966.7	yes	detection + prevention
AEP-full, <code>always</code>	2 063.4	63.4	15.0	yes	detection + prevention
B3-mode, <code>everysec</code>	2 038.2	38.2	0.0	no	detection only

The detection claim of the paper’s outcome table shows no observed difference in any cell, bounded by pooling the capability classes rather than per class, as its detection subsection sets out: it is produced by the pre-dispatch record plus no re-entry, which all three rows have, and the ablation table shows it. What the barrier buys is the last column’s second word, and the prevention subsection is what it is worth. ‘Over floor’ is the same median less the provider’s 2 000 ms delay, and so includes whatever the protocol costs with the barrier already removed – a quantity this evaluation cannot separate from zero: 33.9 ms [-122.6, 120.1] pooled, and 16.3 ms [-54.1, 47.5] over each arm’s lower mode, both spanning zero.

flush, and there are two of them. Nothing in the mechanism is expensive.

The sign is not a puzzle to explain away. The two medians it separates are 2 079.8 ms with the barrier and 2 089.0 ms without it, on a two-second call; the gap between them is noise around a mechanism that is doing no waiting. This arm was pre-registered with the interval’s exclusion of zero explicitly ruled out as an acceptance criterion, because a mechanism with nothing to wait for should cost nothing and a measurement of nothing lands on either side of it.

Both figures are ablation differences *within* one policy, which required collecting the barrier-less arm twice. It would have been easier to subtract the `everysec` B3 median from the `always` AEP-full median, and that would have assumed the ablated protocol’s own writes cost the same under both policies. They nearly do — B3 is 2 038.2 ms under `everysec` and 2 089.0 ms under `always` — but “nearly” is a measurement, and an order-of-magnitude claim resting on an unmeasured premise is the kind of number this paper is trying not to publish.

7.0.0.3 How precise these two figures are: Each is a difference of two medians over fifteen runs per arm, bootstrapped over runs rather than executions. Under `everysec` the barrier costs 1 939.7 ms, [1 855.8, 1 962.4]; under `always` it is -9.2 ms, [-27.2, 44.6]. The first excludes zero by a wide margin and the second contains it, which is the whole of the claim: the policy decides whether the barrier is expensive, and the barrier itself is not.

7.0.0.4 B3-mode: the barrier removed entirely: Not a baseline we are beating but a supported configuration of the same implementation, licensed by the detection result in the paper’s evaluation section. What it pays over a system with no protocol at all is not resolvable from zero at fifteen runs per arm (33.9 ms, [-122.6, 120.1]). On the detection metrics, pooled across capability classes, we observe no difference between it and AEP-full, supported by a bound rather than a test: both the undetected-duplicate and lost-effect rates lie in [0, 4.77%] at joint coverage of at least 90%, and the declared-ambiguity difference lies inside the ± 5 percentage-point margin the paper’s detection subsection states as a post hoc stipulation. *We do not make that claim per capability class, and the reason is a number rather than an oversight:* six simultaneous bounds instead of two, over a third as many run clusters, puts the Bonferroni width at 20.1

percentage points, and a band that wide is not a detection guarantee. What B3-mode forfeits is the prevention result: under a hard Redis kill it will put an effect on the wire that AEP-full withholds, at 0.9333 against 0.3333, and its intent record will not survive a host-level write loss, as the paper’s durability subsection reports. That second clause is about the *record*, and it does not license the protocol-level inference a reader may draw from it: when we ran both arms under exactly that fault, neither withheld, because the storage discarded the writes while reporting success and the barrier was handed an acknowledgement that looked good (the paper’s write-loss cell).

So the question a deployment answers is not “can we afford AEP” but “do we need prevention, and at which fsync policy”. If losing Redis is the fault to worry about — an instance on the same host as the workers, an unreplicated instance, a VM that can vanish — the barrier is the point of the protocol and `always` makes it nearly free at this workload’s shape. If Redis is well protected and worker crashes are the concern, B3-mode gives the paper’s headline result for a cost this evaluation cannot distinguish from zero.

One measurement this table does not support is what `always` costs under load. The Redis documentation calls it “very very slow, very safe” [17], a judgement about write throughput that our workload cannot exercise: two workers, a two-second provider delay and a handful of writes per step leave the system latency-bound and nowhere near an `fsync`-rate limit. Our throughput figure is in fact *higher* under `always` (0.42 versus 0.33 executions/s), which is a statement about this workload’s shape and not about Redis.

8 RQ4: RECOVERY

Recovery behaviour is reported through the outcome distribution rather than a success rate, for a reason worth stating: “recovery success” means two different things either side of whether a recovery service was running. Where one ran, a crashed execution reaching a terminal classification was *recovered*. Where none ran — B0, B1, B2, B4, B4b declare none — the same execution reached its classification because a supervisor ran the step again, which is not recovery but the thing recovery exists to avoid. Our analysis excludes runs without a recovery service from that metric rather than

scoring them zero or one, because both would be statements about a component that was not present.

Within that scope: every AEP-full and B3 execution in the crashed regime reached a terminal classification, and across 432 runs exactly one recorded a reconciliation disagreement — every other completed run’s event log agreed with its independently written oracle ledger. The harness performs this reconciliation after each completed run; it checks definitive applied/non-applied assertions, effect-count bounds, attribution, and duplicate predictions rather than trusting a line count.

That one is worth stating precisely, because the check exists to be believed. It was a B4b cell in which the system classified all ten executions CONFIRMED-APPLIED while the provider’s ledger recorded two, eight disagreements of the form “the system asserted the effect was applied and the ledger records none”. Both sibling runs of the same cell recorded ten, the provider logged no error, and its run-log held three lines against eleven for each sibling — so the provider recorded almost nothing while answering the workers successfully. We could not determine the cause. A re-run of the identical cell, same seed and same configuration, agreed.

We treat it as a *void* run on the rule the durability probe uses: a trial whose measurement precondition failed says nothing about the system in either direction. Counting it would let a broken ground truth set a baseline’s rate; discarding it quietly and keeping the re-run would be re-running until the number looks right. So the re-run is what the paper’s outcome table contains, the voided attempt must ship in the external raw archive under `results/voided/` with this explanation beside it; it is not present in the current Git repository. The count above is one rather than zero. Recovery latency is not reported as an absolute figure: the E5 gate leaves too few gated crashed runs to support a distribution, and the paper’s threats section records this as a gap rather than filling it with ungated numbers.

9 WHY THE PROVABLY-EMPTY CELL IS NOT RESOLVED

The `after_barrier_before_dispatch` row invites an obvious question: if the worker died before sending anything, *the system could know that*. Why is the outcome ambiguous rather than a confirmed non-event?

Because nothing durable records it. A recovery process reading the store afterwards sees an authorized, unresolved intent — exactly what it would see had the worker died *during* transmission, with the request already on the wire. The two histories are identical in everything durable, so recovery fails closed, which is the correct behaviour given what it can observe. **That is the negative result, and it is the protocol’s, not the implementation’s:** the cell is empty in the world and unresolvable from the store. Closing it is possible and we did not do it. The supplementary material gives the probe detail and prices the third barrier that would split the two cases — a trade that narrows rather than closes the window, and whose value depends entirely on a deployment’s ratio of crash points, which our uniform matrix deliberately does not model.

REFERENCES

- [1] C. G. Gray and D. R. Cheriton, “Leases: An efficient fault-tolerant mechanism for distributed file cache consistency,” in *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP)*, 1989, pp. 202–210.
- [2] M. Kleppmann, “How to do distributed locking,” <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>, 2016, accessed 2026-08-07.
- [3] D. Skeen, “Nonblocking commit protocols,” in *Proceedings of the 1981 ACM SIGMOD International Conference on Management of Data*, 1981, pp. 133–142.
- [4] C. Mohan, B. Lindsay, and R. Obermarck, “Transaction management in the R* distributed database management system,” *ACM Transactions on Database Systems*, vol. 11, no. 4, pp. 378–396, 1986.
- [5] H. Garcia-Molina and K. Salem, “Sagas,” in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, 1987, pp. 249–259.
- [6] H. F. Korth, E. Levy, and A. Silberschatz, “A formal approach to recovery by compensating transactions,” in *Proceedings of the 16th International Conference on Very Large Data Bases (VLDB)*, 1990, pp. 95–106. [Online]. Available: <https://www.vldb.org/conf/1990/P095.PDF>
- [7] C. Richardson, “Pattern: Transactional outbox,” Microservices.io, accessed 2026-08-11. [Online]. Available: <https://microservices.io/patterns/data/transactional-outbox.html>
- [8] J. Jena and S. Dalal, “The Idempotency-Key HTTP header field,” Internet Engineering Task Force, Internet-Draft draft-ietf-httpapi-idempotency-key-header-07, Oct. 2025, work in progress; expired 2026-04-18. [Online]. Available: <https://datatracker.ietf.org/doc/html/draft-ietf-httpapi-idempotency-key-header-07>
- [9] S. T. V. Setty, C. Su, J. R. Lorch, L. Zhou, H. Chen, P. Patel, and J. Ren, “Realizing the Fault-Tolerance promise of cloud storage using locks with intent,” in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2016, pp. 501–516. [Online]. Available: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/setty>
- [10] K. M. Chandy and L. Lamport, “Distributed snapshots: Determining global states of distributed systems,” *ACM Transactions on Computer Systems*, vol. 3, no. 1, pp. 63–75, 1985.
- [11] P. Carbone, G. Fóra, S. Ewen, S. Haridi, and K. Tzoumas, “Lightweight asynchronous snapshots for distributed dataflows,” *CoRR*, vol. abs/1506.08603, 2015. [Online]. Available: <https://arxiv.org/abs/1506.08603>
- [12] H. Zhang, A. Cardoza, P. B. Chen, S. Angel, and V. Liu, “Fault-tolerant and transactional stateful serverless workflows,” in *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020, pp. 1187–1204.
- [13] S. Burckhardt, C. Gillum, D. Justo, K. Kallas, C. McMahon, and C. S. Meiklejohn, “Durable functions: Semantics for stateful serverless,” *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, pp. 1–27, 2021.
- [14] Z. Jia and E. Witchel, “Boki: Stateful serverless computing with shared logs,” in *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021, pp. 691–707.
- [15] S. Zhuang, S. Wang, E. Liang, Y. Cheng, and I. Stoica, “ExoFlow: A universal workflow system for exactly-once DAGs,” in *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023, pp. 269–286.
- [16] Temporal Technologies, “Retry policies,” <https://docs.temporal.io/encyclopedia/retry-policies>, 2026, accessed 2026-08-06.
- [17] Redis, “Redis persistence,” https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/, 2026, accessed 2026-08-07.